Artificial Neural Networks

Tjaš Ajdovec

ta5946@fri.uni-lj.si

Introduction

In this homework, we implemented an artificial neural network (ANN) in NumPy for classification tasks, using analytical backpropagation to compute gradients. We then reimplemented it in PyTorch, where gradients are computed using automatic differentiation, and extended the implementation to support regression tasks, multiple activation functions and L2 regularization. We compared both solutions using predicted probabilities, training loss curves, learned weights and training and prediction time. Lastly, we used the PyTorch ANN to classify human tissue based on spectral and spatial information at the pixel level.

Part 1: Neural Network Implementation

First, we built a **fully connected ANNClassifier** in NumPy with sigmoid activations in hidden layers, softmax in the output layer and cross-entropy loss. The network supports any number of hidden layers and neurons per layer. The parameters are stored as a list of weight matrices.

$$\sigma(z) = \frac{1}{1+e^{-z}} \tag{1}$$

$$\frac{d\sigma}{dz} = \sigma(z)\big(1-\sigma(z)\big) \tag{2}$$

The optimization uses **mini-batch gradient descent** for a given batch size and number of shuffled epochs. The forward pass saves the layer activations and calculates the training error. Analytical backpropagation uses the transposed weight matrices to redistribute the error and the sigmoid derivative to scale it at each hidden layer. Xavier weight initialization is used for better training stability.

We verified analytical gradients by comparing them against numerical ones. These were computed by varying each weight by a small ε. The maximum relative differences stayed under 10^{-6}, indicating a correct implementation.

We trained the ANN on the provided datasets to determine the minimal architecture for a perfect fit. For `doughnut.tab`, this was **[3] with 17 weights** and for `squares.tab`, it was **[2, 2] with 18 weights**.

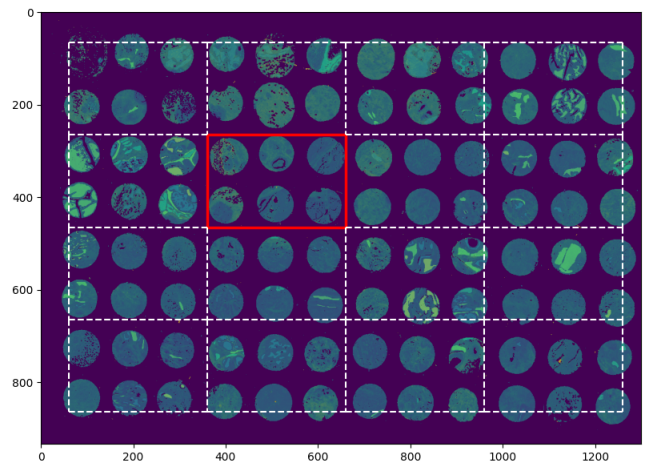

Figure 1. Grid on the spectral data image used for model evaluation. Red rectangle is the competition test set.

Part 2: PyTorch Extension

After that, we reimplemented the ANNClassifier in PyTorch. We extended it to support **regression tasks** with the mean squared error (MSE) loss, more hidden layer activation functions such as ReLU, and optional L2 **regularization** [1]. The gradients are computed by PyTorch autograd when the `loss.backward()` is called.

To match NumPy precision, we set it to `float64`. We trained both implementations on the doughnut dataset. We set the learning rate to 0.5, the number of epochs to 5000, the batch size to 64, the network architecture to [3] and used the sigmoid activation function with no regularization. We observed the following:

- **Predicted probabilities** were almost identical. Both models perfectly fit the training data and achieved 100% classification accuracy.
- **Training loss** curves looked identical.
- **Learned weights** matched up to a precision of 10^{-13}. Pearson correlation between both sets of weights was equal to 1.
- **Training time** was the only noticeable difference. The NumPy implementation fit the dataset in 2.0s, which is

Model	Features	Pool Log Loss	Mean ± Std Log Loss	Mean ± Std Accuracy	Eval Time (s)
LR	spec	0.766	0.822 ± 0.403	0.685 ± 0.208	67.6
LR	**spec + coor**	**0.724**	0.765 ± 0.366	$\mathbf{0.702} \pm 0.197$	50.2
NN	spec	0.775	0.863 ± 0.407	0.681 ± 0.172	18.9
NN	spec + coor	0.694	0.788 ± 0.353	0.712 ± 0.160	21.0
NN	**spec + 3x3 mean + coor**	**0.638**	0.748 ± 0.364	$\mathbf{0.736} \pm 0.163$	27.3
NN	**spec + up to 5x5 mean + coor**	**0.672**	0.835 ± 0.523	$\mathbf{0.737} \pm 0.169$	29.5
NN	spec + up to 9x9 mean + coor	0.781	0.944 ± 0.538	0.714 ± 0.175	41.3

Table 1. Results of model cross-validation (CV) on rectangles in the spectral image. LR stands for logistic regression and NN for neural network. Pooled metrics are calculated per annotated pixel.

5 times faster than its PyTorch equivalent. This was likely due to the small dataset and framework overhead.

The described extended ANN also passed all the provided tests.

Part 3: Spectral Data Classification

Lastly, we were given an image of human tissue with a spectral dimension. The goal was to predict one of six classes for each annotated pixel, specifically within the competition rectangle. To develop and evaluate models, we anchored the image in a grid of such rectangles, as shown in Figure 1. We used these to perform **leave one out CV**.

We used logistic regression with only spectral features as the baseline. We compared it to the ANN classifier with a 2 layer architecture `[128, 64]` and ReLU activation. Other training parameters were also adjusted. The complete CV results are presented in Table 1. **NN based models improved performance** over LR in both pooled log loss, which was our primary metric, and classification accuracy. We introduced additional features by computing mean spectral values over local neighborhoods and extended this to multiple scales. The best performing models achieved a pooled log loss of 0.638 and mean accuracy of 0.737, while also being faster than the baseline.

Besides that, we experimented with different NN architectures and coordinates relative to the rectangle, but this did not clearly improve the results. For the final model, we used a **probability averaging ensemble** of the three best performing models to increase robustness and reduce log loss.

References

[1] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Series in Statistics. Springer, New York, NY, 2 edition, 2009.